

NeuroScan AI - Project File Structure & Guide

This document maps out the entire file structure of the NeuroScan AI application, explaining the purpose of each file and folder.

📁 Root Directory

`c:\Users\faiza\OneDrive\Desktop\scratch\_effnet\_svm\`

File	Purpose
------	---------

-----	-----
-------	-------

run_server.bat	Start Script. Double-click this to launch the backend server (`uvicorn`) and get the app running.
-----------------------	--

test_endpoint_live.py	Connectivity Test. A script to verify that the backend API is reachable and accepting requests.
------------------------------	--

test_analytics.py	Logic Test. A script specifically to debug and verify the `tumor_analytics` and JSON response structure from the ML model.
--------------------------	---

FEATURES_LIST.md	Documentation. A detailed inventory of all implemented features (AI, UI, Tools, etc.).
-------------------------	---

PROJECT_STRUCTURE.md	Documentation. This file. A guide to the codebase layout.
-----------------------------	--

💻 Frontend

`frontend/` - Contains the User Interface code.

Core Files

File	Purpose
------	---------

-----	-----
-------	-------

index.html	Main Application Point. A single-page application (SPA) file that contains the HTML markup, inline critical CSS, and the `render...()` functions for different views (Dashboard, Scan, Report, etc.).
-------------------	--

manifest.json	PWA Config. Metadata to make the app installable as a Progressive Web App (PWA) on mobile devices.
----------------------	---

| **sw.js** | **Service Worker.** Caches assets for offline capability (PWA support). |

Styles (frontend/css/)

| File | Purpose |

|-----|-----|

| **style.css** | **Global Styling.** The main CSS sheet defining the dark theme, glassmorphism effects, utilities, and component styles. |

| **mobile.css** | **Responsive Overrides.** Specific styles to adjust layouts for tablets and mobile screens. |

Logic (frontend/js/)

| File | Purpose |

|-----|-----|

| **app.js** | **Controller Logic.** Handles routing (navigation), form submissions, API calls to the backend, and main state management. |

| **brain_viewer.js** | **3D Visualization.** Uses **Three.js** to render the interactive holographic brain model and the tumor sphere. |

| **diagnostic_tools.js** | **Canvas Tools.** Implements the interactive 2D medical tools: Pixel Probe, ROI Ruler, Filters (Thermal, Edge Detection), and Magnifying Glass. |

| **translations.js** | **Localization.** Contains the dictionary objects for multi-language support (English, Hindi, Spanish, etc.). |

| **api.js** | **API Utilities.** Helper functions for fetching data and handling HTTP requests. |

Backend

`backend/` - Contains the FastAPI server and AI Logic.

Application Core

| File | Purpose |

|-----|-----|

| **main.py** | **Server Entry Point.** Sets up the `FastAPI` app, CORS policies, static file serving, and defines the API routes (endpoints). |

| **database.py** | **DB Connection.** Configures the SQLite database connection (`neuroscan.db`). |

| **models.py** | **Data Models.** Defines the SQLAlchemy database tables (e.g., `Report` table to store patient info and results). |

| **schemas.py** | **Data Validation.** Pydantic models to validate incoming request data (e.g., ensure patient age is a number). |

| **crud.py** | **Database Operations.** Helper functions to Create, Read, Update, and Delete records in the DB. |

AI & Intelligence

| File | Purpose |

|-----|-----|

| **ml_service.py** | **The Brain.** The most critical file. Loads the AI models, processes images, performs **Grad-CAM** heatmap generation, detects bounding boxes, and calculates all **clinical metrics** (size, morphology, etc.). |

| **chat_service.py** | **"Dr. Furqan" Logic.** Handles the conversational AI context, processing user queries about the scan and generating medical persona responses. |

| **efficientnet_feature_extractor.pth** | **Model Weights.** The pre-trained PyTorch parameters for the feature extractor (EfficientNetV2). |

| **efficientnet_svm.pkl** | **Model Weights.** The scikit-learn Support Vector Machine (SVM) classifier that predicts "Tumor" or "No Tumor". |

| **model_swin.keras** | **Secondary Model.** (Optional) Weights for the Swin Transformer model used for consensus. |

| **model_coatnet.keras** | **Secondary Model.** (Optional) Weights for the CoAtNet model used for consensus. |

Utilities

| File | Purpose |

|-----|-----|

| **pdf_generator.py** | **Report Gen.** Uses `reportlab` or similar libs to compile the analysis results into a formatted PDF file. |

| **auth.py** | **Security.** Handles password hashing (bcrypt) and JWT token generation for secure access (if enabled). |

| **requirements.txt** | **Dependencies.** Lists all Python libraries needed (FastAPI, PyTorch, Twilio, etc.). |

| **backend_errors.log** | **Logs.** Captures server errors and stack traces for debugging. |

| **neuroscan.db** | **Database.** The local SQLite file storing all patient reports and history. |

🛠️ Key Directories

| Directory | Contents |

|-----|-----|

| **backend/venv/** | **Virtual Environment.** The isolated Python environment containing all installed libraries. |

| **backend/assets/uploads/** | **Image Store.** Where uploaded MRI scan images are saved. |

| **backend/assets/reports/** | **PDF Store.** Where generated PDF reports are saved. |